

Cowrie | Splunk: en MITRE Engage-operation

Gruppe/navn: Frederik Schwartz

Klasse: 4A e2022 IT-teknolog

Opgave: Afsluttende projekt, IT-teknolog, 4. semester

Vejleder: Dan Madsen

Antal anslag: 47.986



Figur 1: Cowrie

(#72707)

1 Resumé

Med ønsket om at forstå honeypots og hvordan de kan bidrage til sikre netværk, bygges der et miljø til analyse. Miljøet bygges på Google Cloud Platform, og indeholder to virtuelle maskiner med hhv. honeypotten Cowrie + en Universal Forwarder, og Splunk Enterprise. Cowrie indsamler data, sender det til Splunk via Universal Forwarderen, og det visualiseres i et moderne dashboard i Splunk. Hele processen styres af hhv. Kanban og MITREs Engage-framework, og konkluderer at honeypots kan bidrage til sikkerhed ved at informere om hvad der foregår på netværket, og at det er essentielt at visualisere dataen, for at effektivisere arbejdet med dataen. Der er mulighed for at udbygge miljøet med flere honeypots, andre porte, andre protokoller m.m., men stadig ved brug af Splunk og MITRE Engage.

2 Indholdsfortegnelse

1 Resumé.....	2
2 Indholdsfortegnelse	3
3 Indledning.....	6
3.1 Afgrænsning.....	6
4 Indledende undersøgelse	7
4.1 Idéudvikling.....	7
4.2 Research	7
4.2.1 T-pot ved DigitalOcean	7
4.2.2 Cowrie.....	8
4.2.3 Splunk vs. ELK.....	8
4.2.4 Google Cloud vs DigitalOcean	9
4.2.5 MITRE Engage.....	9
5 Metode.....	10
5.1 MITRE Engage	10
5.1.1 Fase 1: Prepare.....	10
5.1.2 Fase 2: Operate	12
5.1.3 Fase 3: Understand.....	13
6 Analyse	14
6.1 Pathfinding.....	14
6.2 Topologi.....	15
6.3 Teori	16
6.3.1 Honeypots.....	16
6.3.2 Google Cloud Platform.....	18
6.3.3 Splunk	19

6.3.4 MITRE ENGAGE.....	19
7 Løsningsdesign (Operate).....	19
7.1 Google Cloud Platform	20
7.1.1 Virtuelle maskiner	20
7.1.2 Firewall-regler	20
7.2 Cowrie	21
7.2.1 Installation	22
7.2.2 Konfiguration.....	23
7.3 Splunk Enterprise	25
7.3.1 Installation	25
7.2.3 Universal Forwarder	26
8 Understand.....	31
8.1 Splunk dashboard.....	31
8.2 Top commands.....	33
8.2.1 Analyse af commands.....	34
8.3 Top lande/byer.....	36
8.3.1 Analyse af lande/byer	37
8.4 Top credentials.....	38
8.4.1 Analyse af credentials.....	38
9 Praktisk projektplanlægning og ledelse (Prepare).....	39
9.1 WBS	39
9.3 Kanban.....	39
10 Konklusion (Understand).....	41
11 Perspektivering (Understand).....	42
12 Litteraturliste	43

13 Figurliste	45
---------------------	----

3 Indledning

I dette projekt tages der udgangspunkt i virkelighedens enterprise-miljøer, ved at forsøge at vurdere et koncept som honeypots i henhold til sårbarhedsvurderinger af et netværk. Som metode benyttes MITRE Engage, som er et framework i arbejdet med deception-teknologi, og også benyttes i "virkeligheden". Fordi Engage hænger så tæt sammen med honeypots, tager projektet form af en MITRE Engage operation fra indledning til perspektivering. Dette fører til problemformuleringen:

Hvordan kan honeypots bidrage til en højere grad af sikkerhed i en virksomheds sårbarhedsvurderinger?

En række underspørgsmål hjælper undervejs til at besvare den:

Hvad er en honeypot, hvad gør den og hvordan opsætter man den?

Hvordan fungerer en honeypot?

Hvordan kan man benytte indsamlet data til bedre at sikre sit netværk og sin arbejdsgang?

Med ønsket om at foretage denne vurdering, tager jeg udgangspunkt i en række moderne, aktuelle teknologier, som hver især bidrager til et lille stykke af ligningen. Der startes med honeypotsamlingen T-POT, som snævres ind til blot at inddrage honeypotten Cowrie, som placeres på Google Cloud Platform. Cowrie modtager SSH- og telnetforbindelser, og logger forbindelser, kommandoer m.m. Disse logs sendes til Splunk Enterprise med en Universal Forwarder, hvor de filtreres og visualiseres i et moderne dashboard. Dette dashboard giver nem adgang til Cowries dataflow, og bruges til at foretage vurderinger om hvad honeypots kan bidrage til i en sårbarhedsvurdering.

Der konkluderes – med hjælp af Engage og Splunk-dashboardet – at honeypots kan bidrage til forøget sikkerhed ved at indhente informationer om hvad angribere foretager sig. Dette kan f.eks. gøres ved at ændre interne politikker og ændre på konfigurationer af de maskiner man har på sit netværk.

3.1 Afgrænsning

I dette projekt arbejdes der ikke med fysiske opsætninger. Emnets natur (at lade angribere komme ind) gør at jeg ikke er interesseret i at løbe risikoen for VM escapes eller andre uforudsete ting. Selvom jeg har tillid til udviklere af honeypots, vil jeg bruge opensource, og arbejdet er altså på eget ansvar.

Derudover tror jeg at fremtiden ligger i cloud (om man er fan eller ej), og derfor vil det komme mig til gode at få erfaring med cloudkoncepter. Desuden kommer jeg uden om problemet med at skaffe fysisk udstyr, såsom firewall (f.eks. pfSense), host, server, kabler m.m. Jeg tror også at jeg ville flytte fokus fra emnet – honeypots – hen på noget andet; nemlig at få et fysisk netværk til at fungere.

4 Indledende undersøgelse

I dette afsnit vil jeg gennemgå de indledende undersøgelser, som ligger til grund for det endelige valg af teknologi og metode.

4.1 Idéudvikling

Som udgangspunkt havde jeg i starten af fjerde semester ikke en fastlåst idé om hvad jeg ville undersøge. Det eneste jeg vidste var at det helst skulle dreje sig om noget IT-sikkerhedsmæssigt. Derfor brugte jeg en del tid på at læse tidligere opgaver på KEA's Opgavebibliotek. En af opgaverne handlede om honeypots. Jeg kendte i forvejen til begrebet, og efter nærmere læsning af opgaven besluttede jeg mig for at vælge honeypots som overordnet teknologi. Opgaven arbejdede med honeypot-samlingen T-POT, så der startede jeg også. I det følgende afsnit vil jeg gennemgå research-processen – altså hvordan jeg går fra idé til konkret plan.

4.2 Research

4.2.1 T-pot ved DigitalOcean

For at få en idé om hvad honeypots handler om, valgte jeg at opsætte en T-POT ved IaaS-udbyderen DigitalOcean. Jeg benyttede en guide til processen (Valeros, 2022), og fik T-POT til at fungere efter hensigten. Her har jeg under resten af projektperioden kunne monitorere aktivitet vha. det indbyggede Kibana-dashboard.

Denne samling af honeypots har således været kørende siden d. 8. oktober, og sidst jeg kiggede (28/11-2022) havde honeypotten Cowrie været udsat for cirka 250.000 angreb. Per dags dato (30/11-2022) fungerer min T-POT ikke længere; måske har al trafikken ødelagt den. Som det kan ses på følgende graf fra DigitalOcean, er der ikke længere aktivitet:



Figur 2: T-POT CPU

(DigitalOcean, 2022)

4.2.2 Cowrie

Som beskrevet, indeholder T-POT-systemet mange forskellige honeypots. En af disse er Cowrie, som monitorerer SSH- og Telnetforbindelser. Jeg har valgt at fokusere på denne af to årsager:

1. SSH og Telnet er velkendte protokoller, som jeg selv kender til. Eftersom jeg er interesseret i selve funktionaliteten af en honeypot, holder jeg mig til disse velkendte protokoller.
2. Cowrie er den næstmest aktive honeypot i T-POT.

4.2.3 Splunk vs. ELK

For at kunne arbejde med dataen Cowrie indsamler, skal det bearbejdes/filtreres/visualiseres. En simpel Google-søgning vil afsløre at der er to populære metoder til formålet: ELK-stack og Splunk. T-POT bruger ELK-stacken, men en søgning på Jobindex viser at Splunk nævnes oftere i jobannoncer. ELK er tre open-source-systemer (Elasticsearch, Logstash og Kibana) (Elastic, 2022) og Splunk er et enkelt "closed-source"-system (Splunk, 2022). Splunk tilbyder to løsninger: Cloud eller Enterprise. Cloud-løsningen har 14 dages prøveperiode med 5 GB data/dag, hvorimod Enterprise-løsningen har 60 dages prøveperiode med 500 MB data/dag.

På grund af Jobindex-søgningen og at der er ét system, vælger jeg Splunk som behandlings- og visualiseringsværktøj. Dertil får jeg brug for at benytte Splunk i længere end 14 dage, så Enterprise-

løsningen med 60 dage er bedst. Det betyder selvfølgelig at jeg skal bruge en platform til at køre mine instanser på. Mit platformvalg gennemgår jeg i det følgende.

4.2.4 Google Cloud vs DigitalOcean

Med udgangspunkt i T-POT mødte jeg DigitalOcean. En IaaS-udbyder i USA, som tilbyder det enhver anden cloududbyder tilbyder. Her kan man oprette "droplets" (aka instanser), som eksempelvis virtuelle maskiner. Desværre er jeg begrænset af priser, og må derfor finde en cloududbyder som har prøveperioder med kredit. Det har Google Cloud (300 USD over tre måneder), og derfor bruger jeg Google Cloud.

4.2.5 MITRE Engage

Fra at jeg fik gang i T-POT var det selvfølgelig klart at jeg skulle arbejde med honeypots. Men hvordan jeg skulle gå fra en opsat honeypot til at svare på min problemformulering stod ikke klart. Det var dog indtil jeg i hacking-simuleringen TryHackMe (TryHackMe, 2022) blev introduceret til MITRE Engage, som er et framework til at arbejde med deception-teknologi, som eksempelvis honeypots og canary tokens. MITRE Engage indebærer 10 iterative trin, som kan benyttes til at forstå og analysere sit netværk i forhold til angreb (MITRE, 2022).

5 Metode

I følgende afsnit vil jeg gennemgå den metode jeg har benyttet i udarbejdelsen af dette projekt. Der er her tale om frameworket MITRE Engage, som er udviklet af organisationen MITRE. MITRE er en bred organisation, som forsker i mange ting, men en af de centrale fokusområder er cybersikkerhed. Herunder falder så Engage, sammen med andre framework, såsom ATT&CK og D3FEND (MITRE, 2022). Hvor ATT&CK og D3FEND fokuserer på hvordan man hhv. forstår sin modstander og forsvaret sig, adskiller Engage sig ved at aktivt at interagere med cyberkriminelle – som f.eks. med honeypots. Engage bruger begrebet ”adversary engagement” (MITRE, 2022), altså at engagere sig i sin modstander, frem for blot at ”gemme sig bag en firewall”.

Dette afsnit skal forstås som et metaafsnit, som indkapsler resten af projektet. Her gør jeg mig overvejelser om kvaliteten og målet med hvordan jeg vil besvare min problemformulering.

5.1 MITRE Engage

Som kort beskrevet tidligere, fokuserer MITRE Engage på interaktion med modstanderen (hackeren/angriberen/indbryderen). Der er tale om et framework, som har ti trin, som fordeler sig i tre kategorier. Processen for dette projekt følger nogenlunde samme fremgangsmåde, og derfor betragter jeg MITRE Engage som min metode (i kombination med WBS og Kanban). Jeg vil i dette afsnit gennemgå de ti trin og de tre faser. Senere i projektet vil jeg referere tilbage til disse afsnit, for på den måde at skabe en rød tråd fra start til slut.

Det er vigtigt at pointere, at MITRE Engage som udgangspunkt er et værktøj til hele IT-afdelinger og potentielt til eksperter. Jeg er studerende på andet år, og har ikke arbejds erfaring med denne slags værktøjer. Derfor har jeg ikke en forventning om at kunne udnytte Engage til det yderste, og der kan være dele af frameworket, jeg enten må omtolke eller udelade. Jeg beskriver blot denne overvejelse, for at justere forventninger til både mig selv og til læseren. Når det så er sagt, tror jeg at der kan komme et godt stykke arbejde ud af denne proces.

5.1.1 Fase 1: Prepare

Som det kan ses fra titlen, skal man først forberede sig. Først og fremmest skal vi definere hvad vi vil – ellers kommer vi ikke langt. Derudover konstrueres et narrativ, der understøtter dette mål, og herunder skal

engagement-miljøet defineres, potentielle aktiviteter skal undersøges og der skal måske konsulteres med andre parter angående acceptable risikoniveauer.

I Prepare-fasen findes der seks trin (MITRE, 2022):

1. "Assess knowledge of your adversaries and your organization"

Eftersom min honeypot vender ud mod det åbne internet på GCP, bliver den udsat for meget. Jeg har en antagelse om at meget af trafikken på det åbne internet er vulnerability scans udført af bots, og altså ikke rigtige mennesker (Obbayi, 2019). Selvfølgelig er der et menneske bag ved botten, men jeg forventer at den primære modstander er bots.

Som beskrevet er jeg ikke interesseret i at udsætte mit eget netværk for denne operation, og derfor ligger honeypotten hos GCP. Operationen udføres af en ikke-professionel, og der er frie tøjler i forhold til konfigurationer. Der er tale om research/en faglig problemstilling, og altså ikke reel produktion med reel risiko.

2. "Determine your operational objective"

Målet med operationen er af faglig karakter – at forstå honeypots. Ydermere handler det om at besvare problemformuleringen:

Hvordan kan honeypots bidrage til en højere grad af sikkerhed i en virksomheds sårbarhedsvurderinger?

3. "Determine how you want your adversary to react"

At modstanderen reagerer på åbne porte ved at forsøge at forbinde, og at forsøge at foretage handlinger ved succesfuld forbindelse.

4. "Determine what you want your adversary to perceive"

Jeg vil gerne have at min modstander skal møde noget der ligner et rigtigt system. Jeg har ikke en forventning om at det skal kunne narre en erfaren black/red-hatter.

5. "Determine channels to engage with your adversary"

Jeg vil gerne interagere via populære protokoller, såsom ssh og telnet, og det skal være via en honeypot.

6. "Determine the success and gating criteria"

Succes opnås ved at jeg får informationer om hvad en honeypot kan og gør, hvordan den fungerer og hvad den udsættes for. Derudover er det en succes hvis jeg får filtreret og visualiseret dataen, så det kan bruges til Understand-fasen. Det er selvfølgelig kun en succes hvis jeg kan svare på min problemformulering.

Nu hvor jeg har gjort mig nogle tanker om forberedelser, kan jeg gå til beskrivelse af Operate-fasen.

5.1.2 Fase 2: Operate

Operate-fasen består ganske "simpelt" af implementering af det miljø, man i *Prepare*-fasen har planlagt. *Operate*-fasen er den mest ressource-intensive, og det er her det meste af arbejdet ligger. MITRE har tre forskellige kategorier inden for *Operate*-fasen, hvortil der kan kobles forskellige elementer og handlinger, som man kan benytte i fasen (MITRE, 2022).

Prepare	Expose		Affect			Elicit		Understand
Plan	Collect	Detect	Prevent	Direct	Disrupt	Reassure	Motivate	Analyze
Cyber Threat Intelligence	API Monitoring	Introduced Vulnerabilities	Baseline	Attack Vector Migration	Isolation	Application Diversity	Application Diversity	After-Action Review
Engagement Environment	Network Monitoring	Lures	Hardware Manipulation	Email Manipulation	Lures	Artifact Diversity	Artifact Diversity	Cyber Threat Intelligence
Gating Criteria	Software Manipulation	Malware Detonation	Isolation	Introduced Vulnerabilities	Network Manipulation	Burn-In	Information Manipulation	Threat Model
Operational Objective	System Activity Monitoring	Network Analysis	Network Manipulation	Lures	Software Manipulation	Email Manipulation	Introduced Vulnerabilities	
Persona Creation			Security Controls	Malware Detonation		Information Manipulation	Malware Detonation	
Storyboarding				Network Manipulation		Network Diversity	Network Diversity	
Threat Model				Peripheral Management		Peripheral Management	Personas	
				Security Controls		Pocket Litter		
				Software Manipulation				

Figur 3: MITRE Engage: matrix

Brugen af Cowrie falder under flere handlinger i denne matrix. Dette betyder ikke at det kun er disse handlinger, der kan bruges i denne forbindelse, men blot at det er relevante for scopet for dette projekt.

Eksempler inkluderer: System Activity Monitoring, Lures, Information Manipulation og Introduced Vulnerabilities. Jeg vil i løbet af projektet referere tilbage hertil, når vi benytter handlinger fra matrixen. Når projektet er implementeret og kørende, kan jeg begynde Understand-fasen, hvor jeg monitorerer, filtrerer og analyserer den data, Cowrie har logget.

5.1.3 Fase 3: Understand

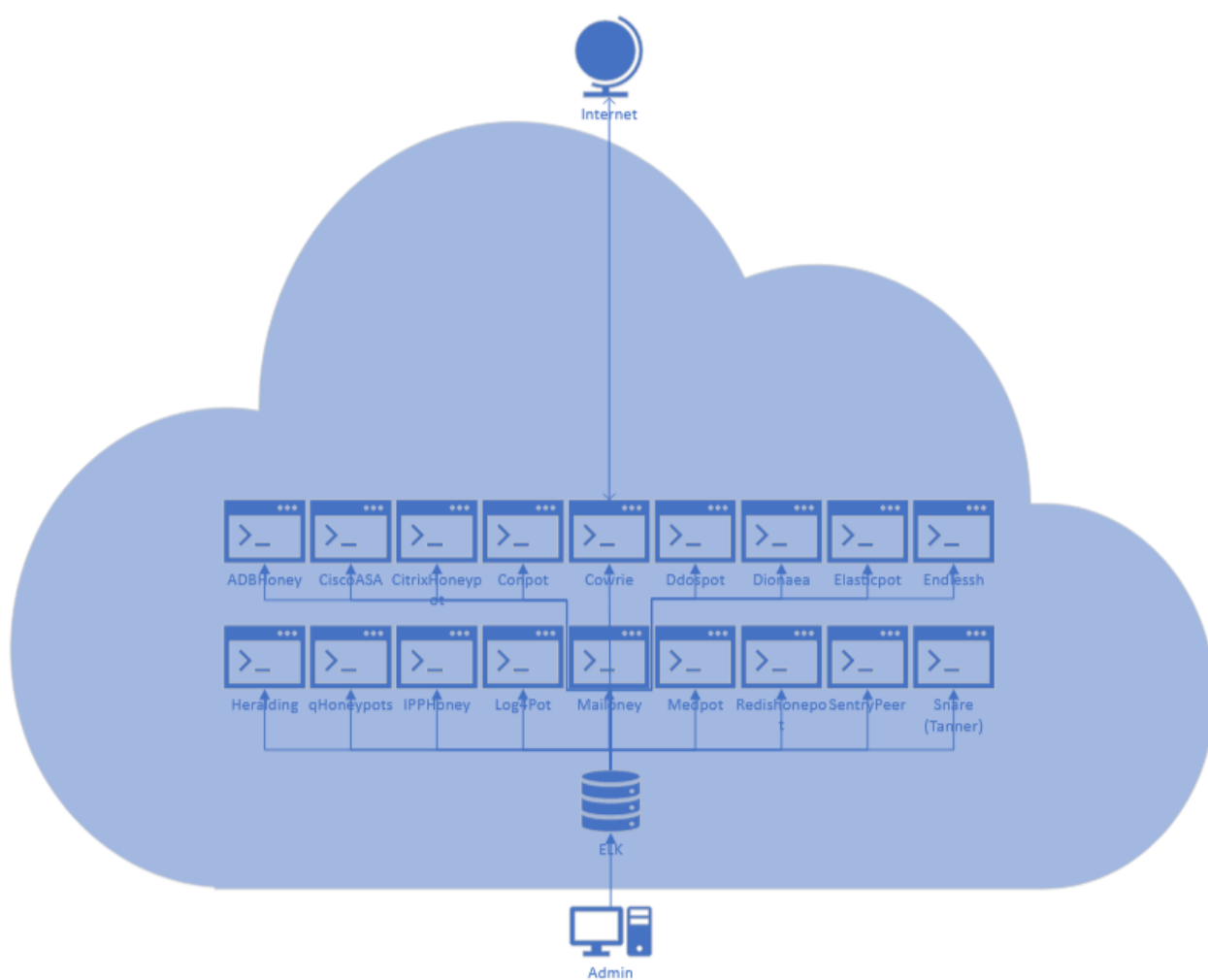
Understand-fasen er en evalueringsfase, hvor man bruger den viden og det data, man indsamler i sin Operate-fasen til at forstå og vurdere om hvorvidt man har opnået sit mål med hele operationen. Eftersom Engage er iterativt, vil det være naturligt i et enterprise-miljø at løbende evaluere og genvurdere hele operationen.

I dette projekts tilfælde bruger vi Splunk, som kan filtrere og visualisere. Det er her jeg som operatør kan skabe overblik over hvad angribere foretager sig, og eventuelt foretage ændringer i konfigurationsfiler på Cowrie.

6 Analyse

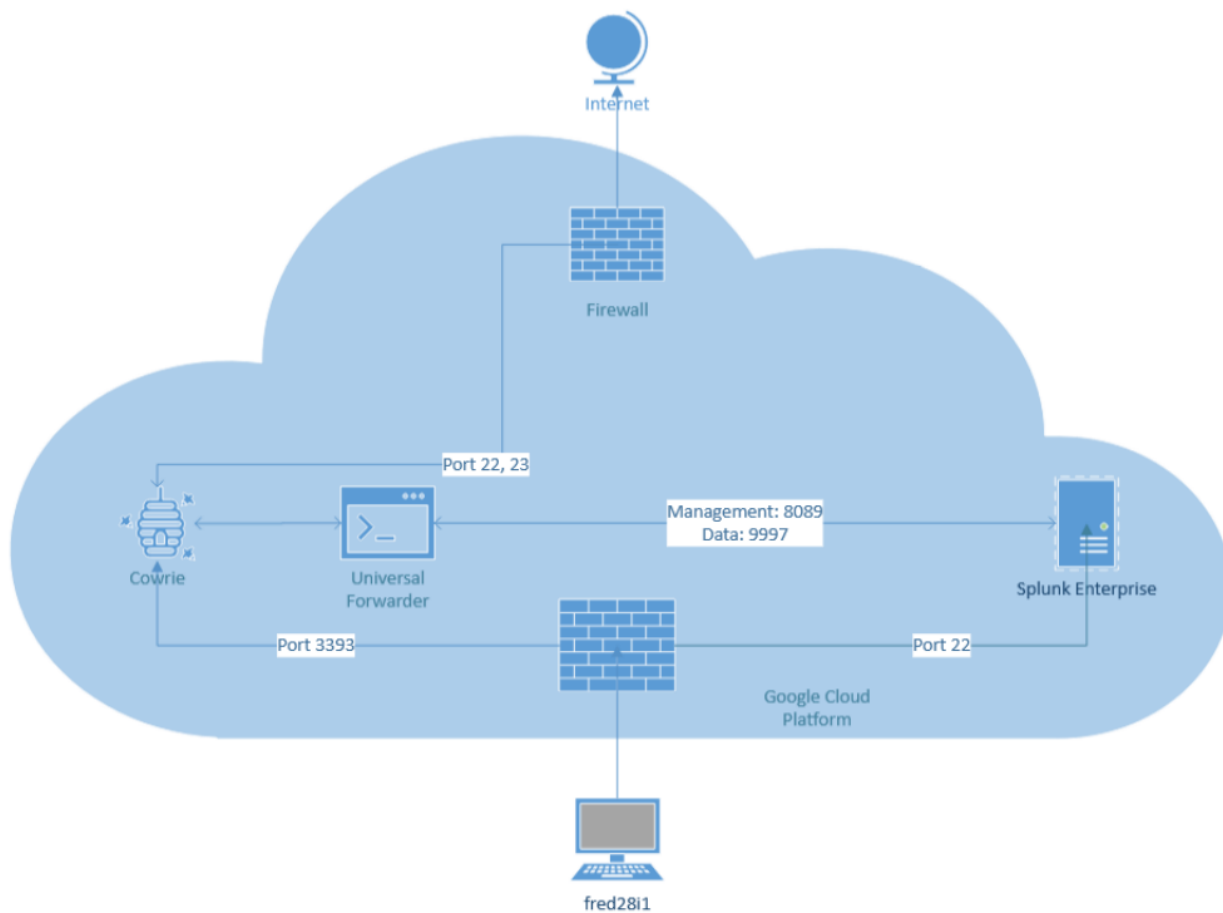
I dette afsnit vil jeg gennemgå de teknologier, som jeg benytter til at besvare min problemformulering, men først pathfinding og topologi for netværk.

6.1 Pathfinding



Figur 4: Topologi v1

6.2 Topologi



Figur 5: Topologi v2 (endelig)

6.3 Teori

I dette afsnit vil jeg gennemgå de komponenter, som indgår i den endelige løsning.

6.3.1 Honeypots

Hvis man arbejder med netværkssikkerhed, møder man ofte firewalls, Access Control Lists, Intrusion Prevention Systems m.m. Hvor disse klassiske komponenter typisk vil blokere angreb, adskiller honeypots sig, ved at tillade angribere adgang til systemet. Denne adgang er kontrolleret af netværkets administrator, og i virkeligheden har angriberen slet ikke adgang til systemet. Honeypots *ligner* rigtige systemer og servere fra angriberens synspunkt, men er i virkeligheden falske systemer, som konfigureres til at se virkelige ud. Honeypots kommer i mange forskellige udgaver. Fra simple "honeyfiles" til hele honeynets til emulerede og virtuelle honeypots på bare metal, rigtige operativsystemer.

I honeypot-litteratur, beskrives honeypottens tilgang til angribere med begrebet "interaction", eller interaktion på dansk. Interaktionsniveauet siger noget om hvor aktiv eller passiv honeypotten er i forhold til angriberen. Ifølge Grimes findes der honeypots med lavt og højt interaktionsniveau (A. Grimes, 2005, p. 14), men andre kilder beskriver også medium interaktionshoneypots (Mahmoud & Pedersen, 2019).

6.3.1.1 Lav interaktion

Honeypots med lavt interaktionsniveau har ret begrænset funktionalitet. Disse honeypots bruger typisk en række services, angriberen kan benytte, men kan være begrænset i hvilke porte og services. Den største fordel er at de er ret nemme at installere, opretholde og derved evt. genopsætte ved kompromitation. Disse honeypots kan typisk indfange data fra worms eller portskanninger, men tillader eksempelvis ikke angriberen at udføre commands i Command Line Interface (CLI) (Mahmoud & Pedersen, 2019, p. 29).

Et meget simpelt eksempel kan være en port listener, som simpelthen åbner en port og alarmerer, hvis porten forsøges forbundet til (A. Grimes, 2005, p. 14).

6.3.1.2 Medium interaktion

Som en lidt mere kompliceret type honeypot, finder vi medium interaktions-honeypotten. Her kan det være nemmere at indsamle mere detaljeret information fra angriberen, som eksempelvis CLI-kommandoer. Medium-interaktionshoneypots kan emulere operativsystemer. Det betyder at angriberen vil se et filsystem, kan udføre kommandoer, logge ind m.m. Fra angriberens synspunkt vil kommandoerne blive

udført, men i virkeligheden er kommandoernes resultat emulerede, og ikke eksekverede. Ydermere opsamles kommandoer og aktivitet i log-filer til senere analyse (Mahmoud & Pedersen, 2019, p. 29).

Ulempen ved disse (og lav-interaktionshoneypots) er at en erfaren angriber relativt let kan finde ud af at der er tale om en honeypot, da funktioner ikke eksekveres:

“Modern honeypots such as Cowrie are still in their infancy but are moderately operative in their default configuration state. This state, however, is not a viable option for security analysts, because Cowrie can easily be detected by attackers. A honeypot is only plausible and effective if its attackers cannot realize that they are actually connecting to a honeypot” (Cabral, Valli, Sikos, & Wakeling, 2019, p. 166)

Derudover er mange af disse honeypots open source, og altså frit tilgængelige på Github m.m., hvilket gør det nemt at undersøge hvordan de ser ud og virker.

6.3.1.3 Høj interaktion

Høj-interaktionshoneypots er rigtige systemer. Rigtige systemer med rigtige operativsystemer, services m.m., og derfor vil de fremstå for en angriber som ægte (hvilket de jo også er). Dette betyder at det vil være meget svært for en angriber at vurdere hvorvidt der er tale om en honeypot – hvis mistanken overhovedet melder sig.

Ulempen ved disse honeypots er naturligvis at der er tale om et rigtigt system. Derfor er der en risiko for at angriberen kan bruge dette system (som vedkommende jo har fået lov at komme ind i) til at foretage yderligere bevægelser på netværket, og potentielt pivottere videre til andre systemer. Derfor er der tale om en slags high risk – high reward-model (Jhummarwal, Panchal, Potdar, & Vadaviya, 2019, p. 352).

6.3.1.4 Opsummering

Honeypots kommer altså – groft opsat – i tre typer, som har fordele og ulemper. Den følgende tabel fra Mahmoud & Pedersen, 2019 viser et rigtig fint overblik (hhv. lav (LIHP), medium (MIHP) og høj-interaktionshoneypots (HIHP)):

	LIHP	MIHP	HIHP
real operating system	no	no	yes
risk of compromise	low	mid	high
wish of compromise	no	no	yes
information gathering	low	mid	high
knowledge to deploy	low	mid	high
knowledge to develop	low	high	high
maintenance time	low	low	very high

Figur 6: Fordele/ulemper v. honeypots

(Mahmoud & Pedersen, 2019, p. 30)

6.3.1.5 Research vs. Produktion

Slutteligt kan der være tale om yderligere to distinktioner mellem honeypots: *research* og *produktion*. Research-honeypots har typisk til formål at undersøge, forstå, forske m.m. i angriberes metoder og teknikker, og ligger *ikke* sammen med produktionen på et netværk. Produktionshoneypots derimod *ligger* sammen med produktionen på netværket, og har til formål at alarmere og dokumentere, *hvis* der skulle være angribere der slipper forbi øvrige sikkerhedsforanstaltninger på netværket (A. Grimes, 2005, p. 39).

6.3.1.6 Cowrie

Som beskrevet tidligere har jeg valgt Cowrie som udgangspunkt for dette projekt. Cowrie er en SSH- og telnet-honeypot, og falder under medium-interaktionshoneypots.

Dette betyder at Cowrie tillader telnet og SSH-forbindelser fra internettet, og at de "besøgende" kan "gennemføre" kommandoer. Cowrie samler alle SSH-forbindelser, telnet-forbindelser, credentials og kommandoer i en logfil på den virtuelle maskine.

Jeg vil i løsningsdesignafsnittet gennemgå installation og funktion af Cowrie.

6.3.2 Google Cloud Platform

Som beskrevet tidligere indebærer der væsentlige risici i en honeypot. Blandt andet kan Cowrie frit forsøges forbundet til fra det åbne internet. Derfor er jeg ikke interesseret i at placere min honeypot direkte på "bare metal" på mit eget eller KEAs netværk.

Google Cloud Platform er en samling komponenter hos Google. Her benytter jeg Google Compute Engine og Google Virtual Private Cloud, til hhv. at oprette de virtuelle maskiner og til at forbinde de virtuelle

maskiner. Google Cloud er således Infrastructure as a Service, eftersom al maskinkraft og netværk ligger på Googles servere (Google LLC., 2022).

6.3.3 Splunk

Splunk er en webbaseret platform, som tillader søgning, analyse og visualisering af maskinegeneret data. Denne data kan komme fra applikationer, websider, sensorer, enheder mv. Dataen kan sendes til Splunk kontinuerligt, og er derfor et godt match til en honeypot. Honeypotten genererer data kontinuerligt i form af .json-filer, og jeg kan altså via Splunk indsamle dataen, og derefter søge i den og visualisere det fremsøgte. I Splunk benyttes Splunks eget sprog: SPL (Search Processing Language). SPL er designet specifikt til Splunks platform, og benytter sig blandt andet af pipelines | og SQL-syntaks til at søge efter data (Splunk, 2022).

Splunk modtager data fra det der hedder en Universal Forwarder, som kort fortalt videregiver data fra den sensor man har installeret den på.

6.3.4 MITRE ENGAGE

De foregående afsnit i del 7 af projektet betragter jeg som grundstenene, der kan understøtte besvarelse af mine underspørgsmål. For at endeligt kunne besvare selve problemformuleringen, skal der tilføjes en analysemodel, der kan give mig et reelt billede af en virksomheds situation. Her vil jeg benytte MITRE ENGAGE, som er et framework til sikkerhedsprofessionelle, der arbejder med "adversary engagement" (MITRE, 2022), altså en form for interaktion med angribere eller "hackere". Disse professionelle kan arbejde med "deception"-teknologi, som eksempelvis honeypots.

Fremgangsmåden består af tre faser: "Prepare", "Operate" og "Understand", og herunder fordeles der ti trin. Jeg vil i del 7 (Løsningsdesign) af dette projekt forsøge at bruge dette framework sammen med mit Google Cloud/Cowrie/Splunk-miljø, for at kunne få en forståelse for sammenhængen, og for at kunne besvare problemformuleringen.

7 Løsningsdesign (Operate)

I dette afsnit gennemgås teknisk materiale, der skal bygges, installeres og konfigureres, for at den endelige løsning kan fungere.

7.1 Google Cloud Platform

Hele løsningen starter på Google Cloud Platform. Der skal oprettes virtuelle maskiner, som kan kommunikere korrekt internt, og der skal oprettes firewall-regler, som dikterer hvem og hvad der kan tilgå miljøet. Som beskrevet benytter jeg Googles gratis prøveperiode.

7.1.1 Virtuelle maskiner

Først opretter jeg en maskine til Cowrie. Denne maskine skal have en rimelig mængde harddisk-plads, da der skal være plads til potentielt mange logs. Jeg sætter 50 GB af til formålet. Google bruger som standard Linux-images; jeg vælger Debian 11 (bullseye), som er den nyeste. Jeg giver denne virtuelle maskine taggene "cowrie" og "uf", for at kunne referere til dem i firewall-opsætningen via Targets.

Dernæst opretter jeg en maskine til Splunk Enterprise. Denne har ikke brug for lige så meget plads, så vi nøjes med 20 GB. Denne maskine får tagget "splunk".

7.1.2 Firewall-regler

Disse virtuelle maskiner er som udgangspunkt isoleret på det Virtuelle Private Cloud-netværk (VPC). For at 1) cowrie skal kunne acceptere forbindelser fra internettet og 2) jeg skal kunne administrere maskinerne, skal der ændres i standard-firewallreglerne.

Følgende regler oprettes i listen over Firewalls på GCP (figur 2):

☐	Name	Type	Targets	Filters	Protocols / ports
☐	allow-ssh-telnet-cowrie	Ingress	cowrie	IP ranges: 0.0.0.0/0	tcp:22, 23
☐	allow-uf	Ingress	splunk, uf	IP ranges: 10.166.0.13, 10.166.0.14	tcp:8089, 9997
☐	default-allow-ssh-splunk	Ingress	splunk	IP ranges: 0.0.0.0/0	tcp:22
☐	http8000	Ingress	splunk	IP ranges: 0.0.0.0/0	tcp:8000
☐	default-allow-ssh	Ingress	cowrie	IP ranges: 0.0.0.0/0	tcp:3393

Figur 6: Firewall-regler

Allow-ssh-telnet-cowrie tillader indgående (Ingress) forbindelser til Cowrie fra alle IP-adresser på port 22 og port 23 (her vil vi gerne have at alle kan komme til). Vi har nu udført Introduced Vulnerabilities fra Engage-matrixen, ved at åbne porte.

Allow-uf tillader indgående forbindelser til Splunk Enterprise og Universal Forwarder (som ligger på Cowrie-VM'en), men kun fra de to adresser. Det betyder altså at de to virtuelle maskiner kun kommunikerer med hinanden på port 8089 og port 9997. Disse to porte bruges af Splunk til hhv. administration og dataudveksling.

Default-allow-ssh-splunk tillader indgående forbindelser fra alle IP-adresser på port 22. Der er her tale om den SSH-forbindelse jeg bruger til installation og administration af Splunk Enterprise. Det er for nuværende nødvendigt med alle IP-adresser, da jeg ikke selv har en fast IP. Dette kan ændres på et senere tidspunkt, hvis nødvendigt.

http8000 tillader indgående forbindelser til Splunk Enterprise fra alle IP-adresser. Det er sådan jeg pt. får mit Splunk-dashboard at se i min browser. Dette kan også ændres til en fast IP-adresse.

Default-allow-ssh tillader indgående forbindelser til Cowrie fra alle IP-adresser på port 3393. Dette er en brugerdefineret ssh-port, som jeg bruger til at administrere Cowrie. *Denne firewall-regel oprettes først efter Cowrie er installeret og konfigureret – ellers er det ikke muligt at SSH'e ind på maskinen.*

Denne opsætning af IP-adresser er bygget med henblik på at gøre det så sikkert som muligt. Det er dog vigtigt at notere at denne del ikke er fokus for dette projekt.

7.2 Cowrie

Cowrie installeres via en SSH-forbindelse fra GCP. Først og fremmest opdateres maskinen med

```
sudo apt-get update
```

Som beskrevet tidligere skal vi have konfigureret ssh til en brugerdefineret port. Dette skyldes, at Cowrie skal bruge port 22 (og port 23) for at kunne modtage forbindelser fra internettet. Hvis vi ikke omkonfigurerede, ville vi blot blive mødt af det falske system, som Cowrie viser angribere. Denne omkonfigurering foregår således:

```
sudo nano /etc/ssh/sshd_config
```

I nano ændrer vi #Port 22 til port 3393, og genstarter ssh-servicen med

```
sudo systemctl restart ssh
```

Vi kan tjekke forbindelsen med

```
sudo systemctl status ssh
```

```
|Dec 06 09:47:51 cowrie-test sshd[4410]: Server listening on :: port 3393.
```

Figur 8: `sudo systemctl status ssh`

Nu kan vi installere Cowrie, og vi kan konfigurere den via port 3393. Cowrie kan ligeledes konfigureres til at lytte på port 22 (som er standarden for ssh-forbindelser, og vil derfor modtage ssh-forbindelsesforsøg).

Dernæst forbereder vi maskinen på at skulle have Cowrie installeret med software-installation:

```
sudo apt-get install git python3-venv libssl-dev libffi-dev build-essential  
libpython3-dev python3-minimal authbind virtualenv
```

Derefter opretter vi en dedikeret Cowrie-konto uden password (med få privilegier), da vi ikke er interesseret i at bruge Cowrie som root (Cowries udvikler anbefaler at undgå dette af sikkerhedshensyn (Oosterhof, 2022)):

```
sudo /sbin/adduser -disabled-password cowrie
```

Systemet anmoder om oplysninger om brugeren; her kan man taste hvad man vil. Til sidst skifter man til denne nye bruger med

```
sudo su cowrie
```

Og går til `/home/cowrie` med

```
cd ~
```

Nu kan vi installere Cowrie

7.2.1 Installation

Vi kloner fra Github:

Git clone <http://github.com/cowrie/cowrie>

Og går ind i den nye mappe: `cd cowrie`

Nu skal vi oprette et virtuelt miljø, for yderligere at isolere Cowrie-installationen fra maskinen, og vi bruger også dette miljø:

```
virtualenv -python=python3 cowrie-env
```

```
source cowrie-env/bin/activate
```

```
cowrie2@cowrie-test:~/cowrie$ source cowrie-env/bin/activate
(cowrie-env) cowrie2@cowrie-test:~/cowrie$ █
```

Figur 9: Virtuelt miljø aktiveres

I dette virtuelle miljø opdaterer vi pip, som skal bruges til at installere requirements fra requirements.txt:

```
pip install --upgrade pip
pip install --upgrade -r requirements.txt
```

Systemet vil nu installere Cowries krav.

7.2.2 Konfiguration

Cowrie kan i høj grad konfigureres, og dette gøres fra cowrie.cfg-filen. Vi får denne fil ved at gå til (vi er stadig i det virtuelle miljø)

```
cd /home/cowrie/cowrie/etc
```

Vi laver en kopi af cowrie.cfg.dist og kalder den cowrie.cfg:

```
cp cowrie.cfg.dist cowrie.cfg
```

Vi ændrer i filen med

```
sudo nano cowrie.cfg
```

Eksempelvis kan man ændre navnet på honeypotten til noget, der ser interessant ud for en angriber. Hostname kan eksempelvis ændres til prod04, så det ligner en produktionsmaskine. I denne fil kan vi yderligere konfigurere vores honeypot efter behov. For nuværende ændrer vi kun på hostnavnet, og aktiverer Telnet ved at ændre

```
Enabled = false -> enabled = true
```

I denne fil kan vi også se, at Cowrie lytter på port 2222 og port 2223 for hhv. SSH og Telnet. Dette vil vi gerne ændre til at vi får data fra port 22 og 23. Dette gøres nemmest med iptables, som kan omdirigere trafik fra en port til en anden. Port-omdirigering fungerer på hele systemet, og derfor skal dette gøres som root (Oosterhof, 2022). Vi afslutter vores virtuelle miljø med kommandoen

```
deactivate
```

Derefter afslutter vi brugen af cowrie-kontoen, og går til vores standardkonto med:

```
exit
```

for til sidst at køre kommandoen

```
sudo su root
```

Nu er vi root og kan bruge iptables til at omdirigere trafik:

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-port 2222
```

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 23 -j REDIRECT --to-port 2223
```

Vi tjekker at det fungerer med

```
iptables -t nat -nvL
```

Og får:

```
root@cowrie-enterprise-v1:/home/fred28il# iptables -t nat -nvL
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source    destination
68752 4101K REDIRECT    tcp  --  *      *       0.0.0.0/0  0.0.0.0/0      tcp dpt:22 redir ports 2222
9084  445K REDIRECT    tcp  --  *      *       0.0.0.0/0  0.0.0.0/0      tcp dpt:23 redir ports 2223
```

Figur 10: iptables -t nat -nvL

Vi omdirigerer altså trafikken som vi vil. Vi afslutter vores root-session med

```
exit
```

..og nu skulle Cowrie være installeret og konfigureret. Nu skal vi blot huske at indsætte firewall-reglen fra GCP, som tillader forbindelser til Cowrie via port 3393, da vi ikke længere kan komme ind via port 22. Til slut skal cowrie startes. Vi går til

```
cd /home/cowrie/cowrie
```

og skifter til Cowrie-kontoen

```
sudo su cowrie
```

og starter Cowrie med

```
bin/cowrie start
```

Vi kan tjekke at den fungerer som ønsket med kommandoen (som kan installeres med `sudo apt-get install net-tools`)

```
netstat -tanpl
```



```
cowrie2@cowrie-test:~/cowrie$ netstat -tanpl
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:2222           0.0.0.0:*               LISTEN      2041/python
tcp        0      0 0.0.0.0:2223           0.0.0.0:*               LISTEN      2041/python
tcp        0      0 127.0.0.1:631          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:3393           0.0.0.0:*               LISTEN      -
tcp6       0      0 :::1:631               :::*                   LISTEN      -
tcp6       0      0 :::3393                :::*                   LISTEN      -
```

Figur 11: netstat -tanpl

Vi skal gerne have to aktiviteter der lytter på port 2222 og port 2223, samt én, der lytter på port 3393.

Cowrie er nu opsat, og lytter på port 22 og 23 (ved at dataen omdirigeres). Når Cowrie opfanger data, lagres det i Cowries log i

```
/home/cowrie/cowrie/var/log/cowrie
```

Her lagres det som en .log-fil og en .json-fil. .json-filerne er en struktureret version af log-filerne, og kan benyttes til at analysere hvad der foregår på honeypotten. Vi vil gerne sende denne data til Splunk, og til det skal man installere det Splunk kalder en Universal Forwarder. Denne Universal Forwarder videresender data til Splunk Enterprise, hvor man kan søge i data, filtrere det og bygge et dashboard til at få overblik over trafik på Cowrie.

Ifølge Engage-matrixen har vi nu opsat *Lure(s)* – og vi er altså igang med at lokke angribere til.

7.3 Splunk Enterprise

Splunk Enterprise skal installeres på den anden virtuelle maskine vi oprettede i GCP. Dette gøres på følgende måde:

7.3.1 Installation

Først skal installationsfilen findes. Den finder man på https://www.splunk.com/en_us/download/splunk-enterprise.html (login krævet), under Linux -> .rpm -> Download. Stop downloaden og vælg Download via Command Line (wget). Vi får nu et komplet link til at downloade filen via command-line. Dette gøres på den virtuelle maskine. Det er ikke så vigtigt hvor filen downloades til, den skal bare ikke ligge i kritiske mapper (bin, etc, usr, opt, var osv.). Vi går til mappen, hvor vi har downloadet til, og kører

```
sudo rpm -i splunk-9.0.2-17e00c557dc1-Linux-x86_64.tgz -C /opt
```

..for at installere i /opt-mappen. Vi går til bin-mappen:

```
cd /opt/splunk/bin
```

Vi kører

```
sudo ./splunk start --accept-license
```

Indtast brugernavn og password (oprettelse). Vi kan også køre kommandoen

```
sudo ./splunk enable boot-start
```

For at sikre at Splunk altid kører. Når installationen er kørt, kan vi åbne en browser (ikke på den virtuelle maskine) og gå til den adresse, GCP har tildelt på port 8000:

Name ↑	Zone	Recommendations	In use by	Internal IP	External IP
cowrie-enterprise-v1	europa-north1-a			10.166.0.13 (nic0)	34.88.11.230 (nic0)
splunkv1	europa-north1-a			10.166.0.14 (nic0)	34.88.80.23 (nic0)

Figur 12: Splunk external IP

Altså 34.88.80.23:8000. Her logger vi ind med de credentials vi lige har oprettet under Splunk-installationen. Nu kan vi bruge Splunk Enterprise, og derfor installere en Universal Forwarder på Cowrie-maskinen.

7.2.3 Universal Forwarder

Splunks Universal Forwarder indsamler data på en enhed, og kan læse logfiler. Denne skal således installeres på den samme maskine som Cowrie, for at kunne sende Cowries data videre til Splunk Enterprise.

Det er vigtigt at huske, at Universal Forwarder som udgangspunkt bruger to porte: 8089 og 9997. Port 8089 benyttes til administration: opdatering, konfiguration, informationssamling m.m. Al data som sendes til Splunk Enterprise foregår som udgangspunkt over port 9997.

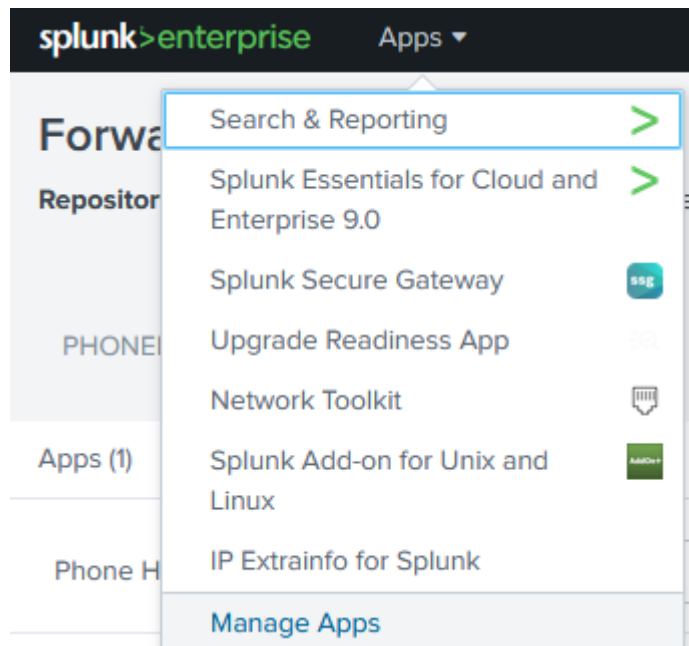
Først skal Splunk Enterprise (SE) konfigureres. Man tilgår SE via IP-adressen fra GCP og port 8000, og logger ind med de credentials man har oprettet. Herefter går man til:

Settings -> Forwarding and receiving -> Under "Receive data": Add new

Indtast "9997" (den port vi bruger til at sende data) og tryk Save.

Dernæst skal vi installere en app i Splunk. Vi går til Apps -> Manage Apps (figur 5).

Herefter trykker vi "Browser more apps", og søger efter "Linux". Nu skal vi installere appen "Splunk Add-on for Unix and Linux". Vi bruger samme credentials, som vi brugte til at downloade Splunk. Denne app har til formål at bearbejde de logs vi får fra Cowrie, ved at strukturere dem, så vi kan arbejde med dem i Splunk.



Figur 13: Splunk app management

Vi har nu konfigureret SE, og installeret en app, som strukturerer dataen fra Cowrie.

Nu åbner vi et nyt browser-vindue, og går til https://www.splunk.com/en_us/download.html, og finder Universal Forwarder (kræver login). Vi vælger Linux, finder .rpm-filen under 64-bit. Vi finder Download via Command Line (wget) og får kommandoen til installationen.

Nu SSH'er vi ind på Cowrie-maskinen via GCP's SSH-funktion (vi bruger selvfølgelig vores brugerdefinerede port 3393):

cowrie-enterprise-v1	europa-north1-a	10.166.0.13 (nic0)	34.88.11.230 (nic0)	SSH	⋮
splunkv1	europa-north1-a	10.166.0.14 (nic0)	34.88.80.23 (nic0)	SSH	

Open in browser window
Open in browser window on custom port

Figur 14: Custom port @ GCP

.. går til /opt og indsætter wget-kommandoen (eventuelt som sudo).

Herefter installerer vi pakken med

```
rpm -i splunkforwarder-9.0.2-17e00c557dc1-linux-2.6-x86_64.rpm
```

Vi starter forwarderen med

```
sudo /opt/splunkforwarder/bin/splunk start --accept-license
```

Den beder om et brugernavn og password, som vi giver den (dette er en oprettelse). Gå til

```
cd /opt/splunkforwarder/bin
```

Kør derefter

```
./splunk set deploy-poll 10.166.0.14:8089 (intern Splunk Enterprise IP-  
adresse:management port)
```

Den beder om credentials fra før. Indtast dem og genstart Splunk UF.

```
sudo ./splunk restart
```

Tilbage til Splunk Enterprise (34.88.80.23:8000), og find Settings -> Forwarder Management, og vi skulle gerne se en forbindelse (det kan godt tage lidt tid før den "phoner home"):

Apps (1) Server Classes (0) Clients (1)

Phone Home: All ▼

All Clients ▼

filter

1 Clients 10 Per Page ▼

i	Host Name	Client Name	Instance Name	IP Address	Actions	Machine Type	Deployed Apps	Phone Home
>	cowrie-enterprise-v1	69EEBD2B-0E6E-4344-A44D-A342145D5228	cowrie-enterprise-v1	34.88.11.230	Delete Record	linux-x86_64	0 deployed	▲ 20 hours ago

Figur 15: Phone home

Nu skal der sendes data til Splunk Enterprise. Dette gøres med følgende kommandoer (fra

```
/opt/splunkforwarder/bin)
```

```
sudo ./splunk add forwarder-server 10.166.0.14:9997
```

Nu skal Universal Forwarderen have at vide hvilke logs, der skal bruges:

```
sudo ./splunk add monitor /home/cowrie/cowrie/var/log/cowrie
```

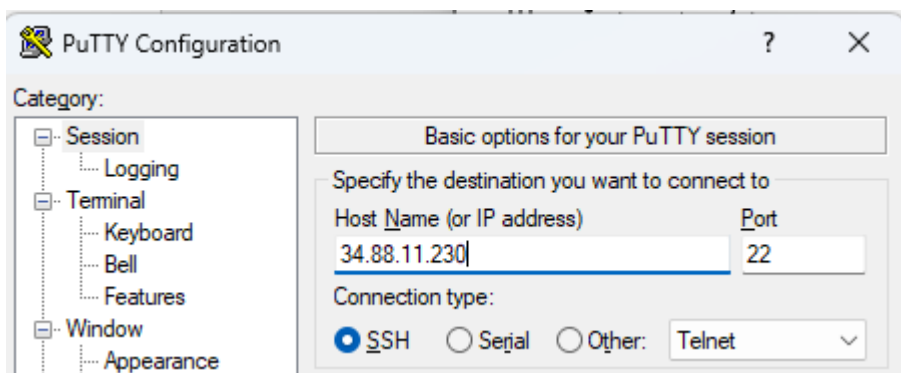
```
sudo ./splunk restart
```

Nu monitorerer Splunk Universal Forwarder vores Cowrie log, og vi kan arbejde med dataen i Splunk Enterprise. Vi går til forsiden på Splunk Enterprise og trykker "Search and Reporting" -> Data Summary -> Sources -> "*.cowrie.json". Vi ser nu at der kommer events ind på Splunk:

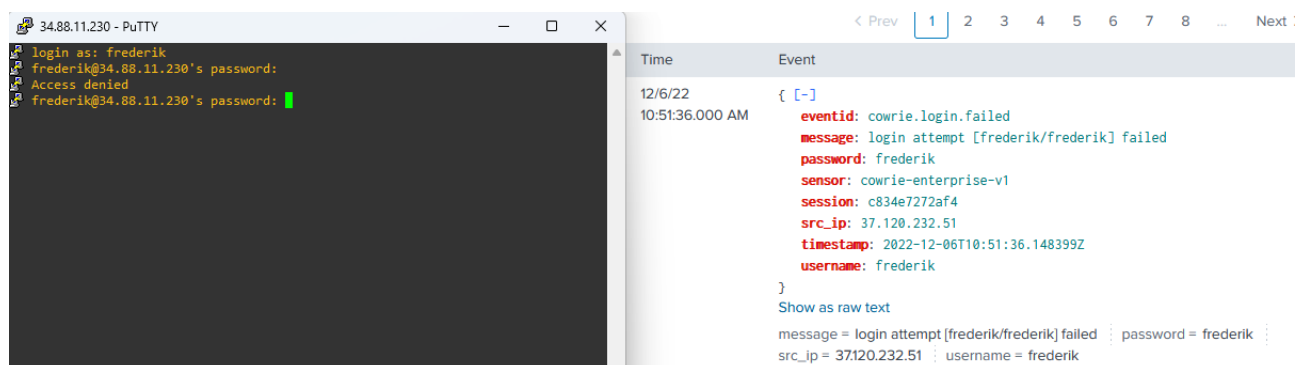
i	Time	Event
> 1	12/6/22 10:45:36.000 AM	<pre>{ [-] duration: 1.703202486038208 eventid: cowrie.session.closed message: Connection lost after 1 seconds sensor: cowrie-enterprise-v1 session: ee0fed8a3a83 src_ip: 129.151.250.217 timestamp: 2022-12-06T10:45:35.808173Z }</pre> Show as raw text message = Connection lost after 1 seconds src_ip = 129.151.250.217
> 2	12/6/22 10:45:36.000 AM	<pre>{ [-] eventid: cowrie.login.failed message: login attempt [wasadmin/wasadmin] failed password: wasadmin sensor: cowrie-enterprise-v1 session: ee0fed8a3a83 src_ip: 129.151.250.217 timestamp: 2022-12-06T10:45:34.706321Z username: wasadmin }</pre> Show as raw text message = login attempt [wasadmin/wasadmin] failed password = wa

Figur 16: Events i Splunk

Vi vil selvfølgelig gerne sikre os at det rent faktisk er data fra Cowrie, og det gør vi ved at prøve at tilgå Cowrie via SSH eller Telnet fra en anden maskine. Vi bruger PuTTY til dette formål:

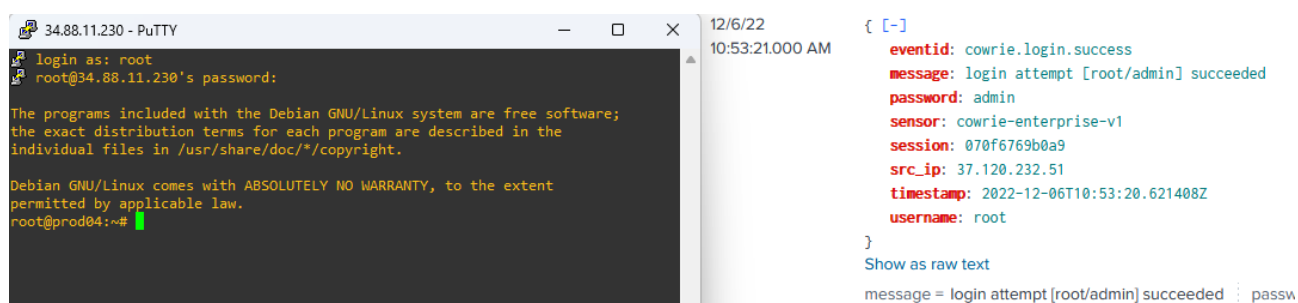


Figur 17: PuTTY ssh



Figur 18: Forsøgt login

Her har jeg forsøgt at logge ind med frederik:frederik. Det er mislykket, og Splunk har fanget eventet. Vi kan også forsøge med root:admin, som pt. godkendes hos Cowrie. Vi kan se at jeg er logget ind på prod04 (som vi tidligere ændrede i konfigurationsfilen).



Figur 19: root:admin godkendt

Vi kan ligeledes forsøge at logge ind via Telnet fra PuTTY:

_time	dst_ip	dst_port	message	password	protocol	src_port	src_ip	username
12/6/22 10:59:56.000 AM			login attempt [frederik/frederik] failed	frederik			37.120.232.51	frederik
12/6/22 10:59:52.000 AM	10.166.0.13	2223	New connection: 37.120.232.51:45028 (10.166.0.13:2223) [session: c14448dd0f4d]		telnet	45028	37.120.232.51	

Figur 20: Telnet-forbindelse

Vi vil også gerne sikre os at vi rent faktisk er logget ind på en honeypot, og ikke den rigtige maskine. Derfor kører vi kommandoen

```
uname -a
```

for at få viden om den "maskine" vi er logget ind på. Denne kan vi så sammenligne med den rigtige maskine, ved at bruge port 3393 fra GCP:

```
fred28i1@cowrie-enterprise-v1:~$ uname -a
Linux cowrie-enterprise-v1 5.10.0-19-cloud-amd64 #1 SMP Debian 5.10.149-2 (2022-10-21) x86_64 GNU/Linux

root@prod04:~# uname -a
Linux prod04 3.2.0-4-amd64 #1 SMP Debian 3.2.68-1+deb7u1 x86_64 GNU/Linux
```

Figur 21: `uname -a` sammenligning

Som vi kan se får fred28i1@cowrie-enterprise-v1 det rigtige navn og den rigtige version af Linux, hvorimod root@prod04 får nogle andre informationer.

Vi har nu bekræftet at Cowrie opsamler SSH- og Telnet-forbindelser, at Splunk modtager dataen samt at Cowrie viser angribere et falskt system.

8 Understand

Vi er nu nået til tredje fase af Engage-frameworket, Understand-fasen. Vi har planlagt og gennemført en operation, og nu skal vi undersøge hvad vi har fundet ud af, og finde ud af om man kan gøre noget anderledes. Først og fremmest vil jeg gennemgå det visualiseringsdashboard, jeg har bygget i Splunk. Vi bevæger os nu til trin otte i Engage, altså

“8. Turn raw data into actionable intelligence” (MITRE, 2022)

8.1 Splunk dashboard

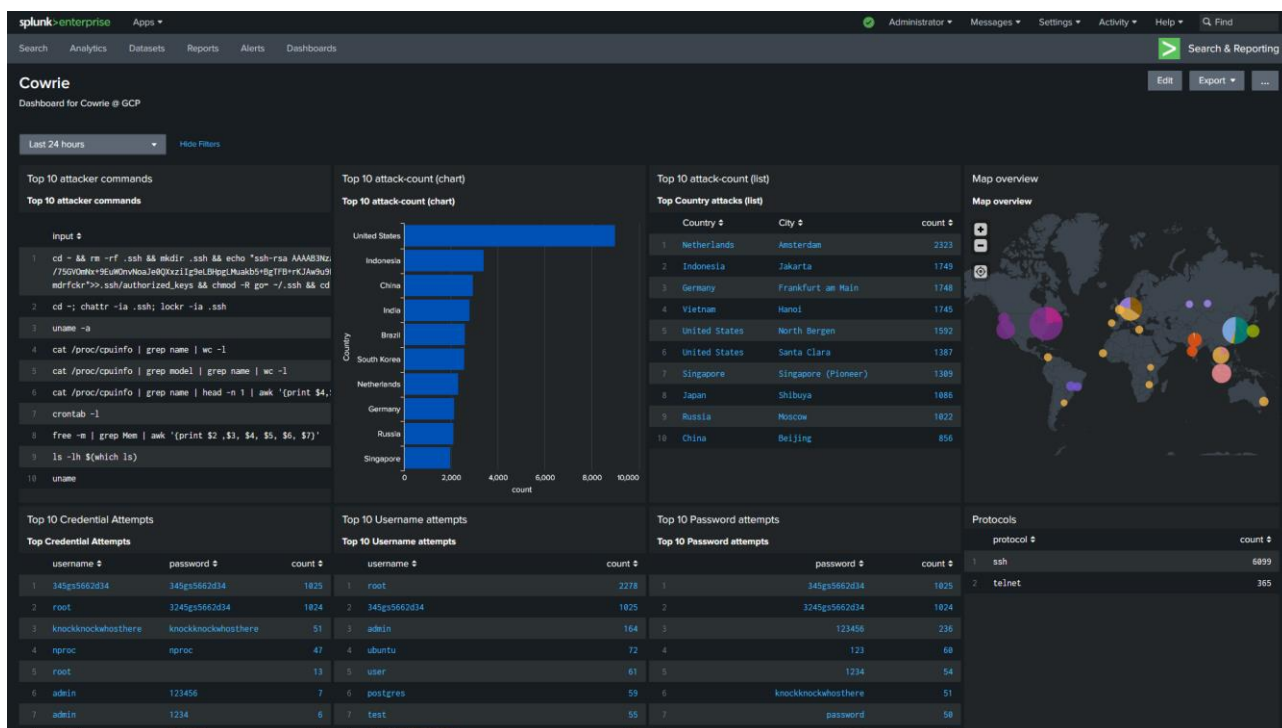
I et miljø hvor man skal forsøge at få en forståelse for aktivitet, og eventuelt reagere på hvad man finder, kan det være voldsomt besværligt og uproduktivt at skulle kæmpe sig igennem logfiler manuelt – uanset hvor strukturerede de end er. Derfor er det fornuftigt med en oversigt, hvor man hurtigt kan få et overblik, før man eventuelt går i dybden med enkelte logs. Dette kan Splunk heldigvis hjælpe med.

Som beskrevet tidligere bruger Splunk pipelines i søgninger, hvilket gør det meget nemt at finde den data man skal bruge. Det er nemt at sammensætte søgeresultater i én sætning, som man kan bruge til at visualisere sin data i et dashboard. Jeg har pt. syv søgninger visualiseret på mit dashboard, og jeg vil nu gennemgå nogle af dem. Søgninger er bundet op på tidspunkt, så der kan søges i eksempelvis hele Cowries levetid eller i real-time. Der ligger en fremvisning af mit dashboard på følgende YouTube-link:

<https://www.youtube.com/watch?v=5hvXtmX6T5s>

eller forkortet link: shorturl.at/ensv9

Screenshot af mit dashboard:



Figur 22: Splunk dashboard

Jeg vil nu gennemgå tre interessante vinkler fra Splunk-dashboardet: de mest populære command line-kommandoer, lande hvorfra der kommer flest angreb og de mest populære credentials/brugernavn + password-kombinationer.

8.2 Top commands

Hvad foretager angribere sig når de kommer ind på netværket/systemet? Som beskrevet, logger Cowrie kommandoer, som angribere foretager. Kommandoerne bliver ikke eksekverede, men for angriberen ser det sådan ud. Denne søgning er rigtig interessant for at få viden om hvad man skal være bevidst om, når man opsætter sit system. Søgningen ser således ud:

CMD: | table input message src_ip | stats count by input | sort -count | head 10

CMD:	table input	stats count by input	sort -count	head 10
Cowrie sætter dette mærkat på alle kommandoer der indsættes. Derfor kan vi filtrere andet fra med denne søgning.	Cowrie logger kommandoer (input). Her laver vi en tabel med disse	Vi tæller antallet af hvert enkelt input, for at kunne sortere dem efter antal	Her sorterer vi inputs efter antal. Alternativt kan man vælge +count, for at få de mindst populære inputs	Vi vælger top 10 for at begrænse antallet i dashboardet

Top 10 attacker commands	
input ↕	count ↕
1 <code>uname -a</code>	5467
2 <code>cat /proc/cpuinfo grep name wc -l</code>	5300
3 <code>cat /proc/cpuinfo grep name head -n 1 awk '{print \$4,\$5,\$6,\$7,\$8,\$9;}'</code>	5283
4 <code>free -m grep Mem awk '{print \$2 , \$3, \$4, \$5, \$6, \$7}'</code>	5283
5 <code>cat /proc/cpuinfo grep model grep name wc -l</code>	5281
6 <code>crontab -l</code>	5281
7 <code>ls -lh \$(which ls)</code>	5281
8 <code>which ls</code>	5281
9 <code>w</code>	5279
10 <code>lscpu grep Model</code>	5278

Figur 23: Top attacker commands

8.2.1 Analyse af commands

Nu har vi samlet data, filtreret den og skabt en overskuelig måde at analysere dataen på. I forhold Engage-matrixen arbejder vi med System Activity Monitoring her. Trin 9 af Engages fremgangsmåde beskrives som følger:

“The actionable intelligence gained from the operation must be fed back into existing models to inform future decisions. Whenever these models are updated, it is essential to revisit any operational decisions that were made using the old intelligence.” (MITRE, 2022, p. 26)

Tanken er altså her at vi analyserer dataen, og efterfølgende sender vores konklusioner tilbage i de fremgangsmåder, organisationen i forvejen har. Nu er jeg jo ikke en hel organisation, og har blot til hensigt at få en forståelse for honeypots, men jeg synes alligevel at der er visse ting vi kan tage med videre.

Eksempelvis har vi i opsætningen af Cowrie ikke udnyttet mulighederne for brugerdefinition i konfigurationsfilen. Her er der mulighed for yderligere at gøre Cowrie mere troværdig, i tilfælde af et rigtigt menneske forbinder – og derved udfører vi System Manipulation fra Engage-matrixen. Eksempelvis kan vi ændre på hvilket system, Cowrie imiterer:

```
# Fake architectures/OS
# When Cowrie receive a command like /bin/cat XXXX (where XXXX is an executable)
# it replies with the content of a dummy executable (located in data_path/arch)
# compiled for an architecture/OS/endian_mode
# arch can be a comma separated list. When there are multiple elements, a random
# is chosen at login time.
# (default: linux-x64-lsb)

arch = linux-x64-lsb
```

Figur 24: Cowrie architecture

Vi kunne også ændre i kernel_versionen. Som det kan ses i figur x, ændrer vi her responsen, når angriberen udfører uname-kommandoen:

```
# Modify the response of '/bin/uname'
# Default (uname -a): Linux <hostname> <kernel_version> <kernel_build_string> <hardware_platform> <operating system>
kernel_version = 3.2.0-4-amd64
kernel_build_string = #1 SMP Debian 3.2.68-1+deb7u1
hardware_platform = x86_64
operating_system = GNU/Linux
```

Figur 25: Cowrie kernel

Hvis vi refererer tilbage til 10.1 Top commands, ser vi at netop denne kommando (uname -a) er den absolut mest populære kommando udført i Cowries falske system.

Disse to ændringer er relativt simple, og vil sandsynligvis ikke narre en erfaren black-hatter. Derfor kan det være en mulighed at ændre hvilken kommandoer, en angriber kan udføre. Cowrie er opsat, så den kan imitere PowerShell-kommandoer, og disse kan ændres efter behov.

```
# File that contains output for the 'ps' command.
#
# (default: share/cowrie/cmdoutput.json)
processes = share/cowrie/cmdoutput.json
```

Figur 26: Cowrie PowerShell

PowerShell er meget populært, så det kunne være relevant at ændre i denne fil. I filen finder vi en lang liste over ps-kommandoer. Ifølge (Cabral, Valli, Sikos, & Wakeling, 2019, p. 169) kan Cowrie imitere en fuldstændig PowerShell-session. Det er ikke i scopet for denne opgave at gennemgå disse kommandoer, men her er der potentiale for virkelig at bygge et system der ser ægte ud.

Generelt handler kommandoerne som udgangspunkt om at samle informationer om det system, angriberen forbinder til: cat /proc/cpuinfo giver informationer om CPU'en – måske mere cryptominingpotentiale? crontab -l undersøger hvilke cronjobs, der kører. ls -lh giver et overblik over filer og de tilladelser, de har, og top giver informationer om hvilke services og systemer der kører. Mange af angrebene kan altså kategoriseres som "Active Reconnaissance" – altså at samle informationer om offeret (Dunkerly, 2021, p. 75).

8.3 Top lande/byer

Det er også interessant at se trends for hvilke lande angreb kommer fra, på hvilke tidspunkter, tidszoner m.m. Det gør vi med:

source="/home/cowrie/cowrie/var/log/cowrie/*.json" | iplocation src_ip | stats count by Country City | table Country City count | sort -count | head 10

source="/home/cowrie/cowrie/var/log/cowrie/*.json"	iplocation src_ip	stats count by Country City	table Country City count	sort -count	head 10
I dette tilfælde vælger vi hele loggen fra Cowrie som source. Det var ikke nødvendigt i den tidligere søgning, da Cowrie automatisk giver loggen et keyword ("CMD:")	Splunks indbyggede funktion "iplocation" kan med data i IPv4-format. Denne funktion bruger GeoLite2-databasen til at finde ud af hvor IP-adressen "kommer fra"	Vi tæller hvor mange gange en land/by-kombination forbindes til en IP-adresse	Vi laver en tabel over lande, byer og antal events	Vi sorterer antallet af events	Vi vælger top 10

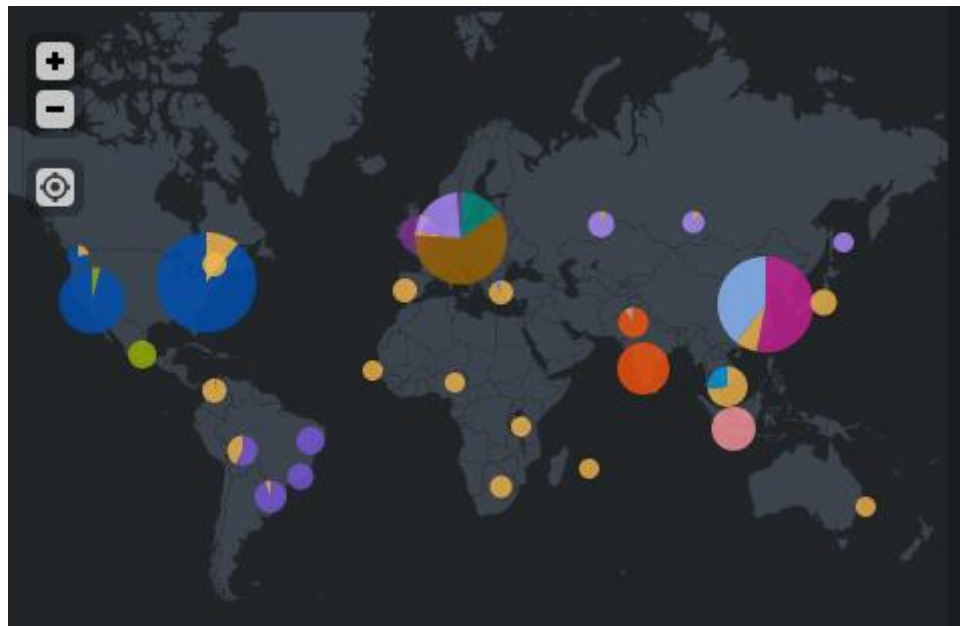
	Country	City	count
1	Singapore	Singapore (Pioneer)	27938
2	United States	North Bergen	21041
3	Germany	Frankfurt am Main	19807
4	India	Bengaluru	12634
5	United States	Santa Clara	12292
6	Netherlands	Amsterdam	9758
7	Singapore	Singapore	9214
8	United States	Clifton	8810
9	China	Haidian (Haidian Qu)	8186
10	Indonesia	Jakarta	7612

Figur 27: Top countries/cities

8.3.1 Analyse af lande/byer

Vi har lagt Cowrie hos Google Cloud Platform. Derfor giver det mening at USA repræsenteres ofte i top 10. Derudover kommer angrebene fra det meste af verden: Asien, Europa m.m. Dette er selvfølgelig de mest populære, og en hurtig redigering i søgningen viser at der er angreb fra stort set hele verden. Jeg ændrer blot til top 1000 (head 1000), og ser nu også angreb fra Mellemøsten, Australien, Sydamerika m.m. Dette kan også ses på kortet, som bruger samme data, men blot visualiserer det (figur x). Der er dog tre større koncentrationer:

Nordamerika, Europa og
Østasien.



Figur 28: Kort over forbindelsesforsøg

8.4 Top credentials

Vi kan undersøge hvordan angribere forsøger at logge ind, og derved få informationer om hvilke kombinationer af credentials, der absolute *ikke* skal bruges:

```
source="/home/cowrie/cowrie/var/log/cowrie/*.json" | stats count by username password | sort -count | head 10
```

source="/home/cowrie/cowrie/var/log/cowrie/*.json"	stats count by username password	sort -count	head 10
Vi vælger hele loggen	Vi tæller antallet af unikke brugernavne og passwords	Vi sorterer antallet	Vi vælger 10 top

8.4.1 Analyse af credentials

Vores dashboard giver os et godt indblik i hvordan angribere forsøger at logge ind. Mange af disse credentials kan vi se er typiske default-credentials (admin, root), men også typisk dårlige credentials (test, 123, 123456, user) og mere specifikke credentials (nproc, pi).

Kommandoen nproc er dog klart den mest populære. Ved nærmere undersøgelse, hænger disse credentials sammen med "cryptojacking", altså forsøg på at overtage maskiner til at udføre cryptomining for angriberen (Bell, 2021).

Top 10 Credential Attempts		
Top Credential Attempts		
	username ↕	password ↕
		count ↕
1	nproc	nproc
2	admin	admin
3	root	root
4	root	admin
5	root	1234
6	test	test123
7	admin	1234
8		
9	ftpuser	ftpuser1234
10	pi	raspberry

Figur 29: Top credentials

Slutteligt er det værd at nævne at dashboardet som det kan ses giver et rigtig godt indblik i hvilke credentials der forsøges logget ind med. Jeg tænker at dette kunne være et effektivt værktøj til at håndhæve password-politikker på Enterprise-niveau. Hvis man kan vise medarbejdere helt konkrete login-forsøg, tænker de måske sig om en ekstra gang, når de vælger passwords.

Som det kan ses er det relativt simpelt at finde det data man skal bruge, når det er struktureret, som vi gjorde i Splunk Forwarder-opsætningen med appen. I det følgende afsnit vil jeg beskrive hvordan min metode hjælper mig til at besvare min problemformulering.

9 Praktisk projektplanlægning og ledelse (Prepare)

For at strukturere mit arbejde har jeg benyttet to fra studiet kendte værktøjer: WBS og Kanban. Jeg vil kategorisere disse værktøjer under Prepare-fasen i Engage, da det jo er her jeg danner overblik og får sat struktur på hele projektet.

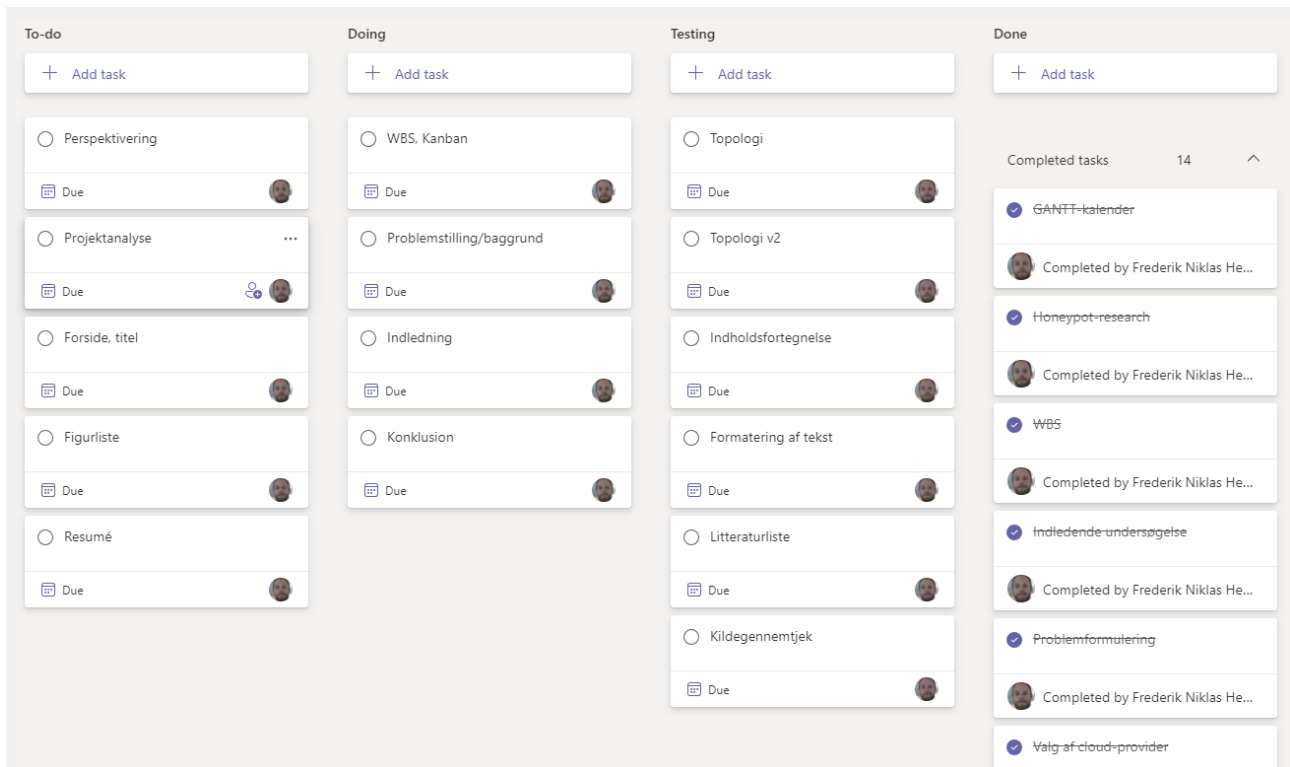
9.1 WBS

I et forsøg på at skabe overblik og forsimple opgaverne, har jeg brudt opgaverne ned i så små dele som muligt, så det stadig giver mening. På den måde kan jeg tage dele en ad gangen, og undgå at miste overblikket. Man leder i en WBS-proces typisk efter en balance mellem konkrethed og overskuelighed. Jo mere konkret, jo nemmere er det at gå igang med opgaven; men mere opdelt og konkrete opgaver resulterer også i *flere* opgaver (Lassen & Olsen, 2019, p. 129). Flere opgaver kan hurtigt blive uoverskueligt, og især når man som jeg skriver alene.

Nedbrydelsen af opgaver har ført til 26 opgaver. Det grænser sig til uoverskuelighed, så jeg har måttet strukturere disse. Det har jeg gjort med et Kanban-board, som jeg vil gennemgå i det næste afsnit.

9.3 Kanban

Kanban er et populært projektværktøj, som benytter kategorier hvori man kan placere det man finder i f.eks. en WBS-proces. Man bygger et Kanban-board, hvor man opsætter kategorier efter behov (Radigan, n.d.). Jeg har opsat fire kategorier: To-do, Doing, Testing og Done. Disse fire buckets er meget simple, men har hjulpet til ikke at miste overblikket. Det har gjort processen simplere, og det er nemt at flytte opgaver (eller *cards*) fra kategori til kategori, som de ændrer sig.



Figur 30: Kanban-board

Jeg har benyttet Microsoft Teams til at opsætte mit Kanban-board, som er integreret i OneDrive og med kalender. På den måde har jeg kunnet samle alting ét sted, istedet for at have fire websider jeg skal tilgå for at styre mit projekt. I skrivende stund (12/12-2022) ser mit board således ud:

Som det kan ses, er de fleste opgaver færdige, og det resterende i To-do-listen drejer sig om mere projekttekniske ting. I tidligere projekter har jeg arbejdet med andre, og her har vi benyttet Scrum. Det gav ikke mening for mig at bruge Scrum, da man typisk har forskellige roller (evt på skift) – det giver ikke mening, når man skriver alene.

10 Konklusion (Understand)

Som led i Understand-fasen i Engage, vil jeg nu besvare min problemformulering:

Hvordan kan honeypots bidrage til en højere grad af sikkerhed i en virksomheds sårbarhedsvurderinger?

Før man begynder at implementere honeypots er det selvfølgelig nødvendigt at gøre sig nogle tanker om operationen inden (*Prepare*). Her er det vigtigt at beslutte sig for hvilken type honeypot man er interesseret i, hvor mange, og hvor de skal placeres – samt hvad formålet er. Eksempelvis skal man vurdere hvor højt et interaktionsniveau man er interesseret i – samt medvurdere de risici der medfølger. Hertil kommer risikoen i placeringen: er der tale om en research- eller produktions-honeypot? Hvor skal den placeres på netværket?

Uanset hvad man kommer frem til, kan honeypot bidrage til højere sikkerhedsniveau, ved at samle informationer om hvad angriberen laver. Hvilke commands, hvorfra, adgangsmetode m.m. Disse informationer kan man benytte til at sikre sig imod disse ting, ved at øge bevidstheden blandt medarbejdere, blokere relevante porte og IP-adresser samt sikre sig imod ødelæggende commands.

For at alt dette skal fungere (i hvert fald så det er nogenlunde overskueligt), er det en rigtig god idé at strukturere og visualisere sin data. Det er ikke konstruktivt at sidde og gennemlæse rå logfildata i /data på din Linux-maskine. Derfor er behandlingen af dataen central for at kunne øge sikkerheden. Det kan man gøre på flere måder, men et eksempel er Splunk. Ved at sende data fra sin honeypot til Splunk via en Universal Forwarder, kan man skabe et overblik via et dashboard. Med et dashboard kan man få de vigtigste informationer hurtigt, hvilket kan være rart i en travl hverdag.

Slutteligt er det givende at pakke det hele ind i et framework såsom MITRE Engage, således at hele processen er struktureret og at kollegaer kan tage over. Dette framework gør det også nemmere at finde løsninger på problemerne, ved brug af Engage Matrixen, som knytter sig til MITRE ATT&CK.

11 Perspektivering (Understand)

Der er meget potentiale i honeypots, og i dette projekt har jeg blot skrabt lidt i overfladen. Som det kan ses i Research-fasen, mødte jeg T-POT, som er en samling af mange honeypots. Alle disse honeypots kan forskellige ting og kan konfigureres på andre måder: andre porte, andre protokoller, andre måder at snyde sin modstander.

Man kan selvfølgelig også vælge at bruge flere honeypots (Cowrie eller ej), og placere Universal Forwarders på dem alle, og sende det hele til Splunk, hvor man kan skabe ét stort dashboard – lidt som T-POT har med ELK-stakken.

Yderligere kan man vurdere sin risikolyst fremover (med grundig research inden) og udforske høj-interaktionshoneypots. Her arbejder man med et rigtigt styresystem, og måske kan man være "heldig" at få et menneske lokket i fælden? Man kan også gå den anden vej, og udforske lav-interaktionshoneypots – men på et rigtigt netværk. Igen følger der en risiko med, men omvendt kan potentielt også få anden data, som kan være mere givende.

Når det så er sagt, tror jeg stadig at Splunk og MITRE Engage ville være elementer jeg ville benytte. Dermed ikke sagt at der ikke er andre muligheder, men jeg kan godt lide at arbejde med aktuelle og relevante systemer.

12 Litteraturliste

- #72707. (n.d.). Image of tiger cowrie (*Cypraea tigris*) on a white background. Undersea Animals. Sea shells.
Image: #56338986. Colourbox.
- A. Grimes, R. (2005). *Honeypots for Windows*. New York: Springer-Verlag.
- Bell, S. (2021, juli 22). *Cryptojacking Attacks Continue To Target SSH Servers*. Retrieved november 30, 2022, from securehoney.net: <https://securehoney.net/blog/cryptojacking-attacks-continue-to-target-ssh-servers.html>
- Cabral, W. Z., Valli, C., Sikos, L. F., & Wakeling, S. G. (2019). Review and Analysis of Cowrie Artefacts and Their Potential to be used Deceptively. *2019 International Conference on Computational Science and Computational Intelligence (CSCI)* (pp. 166-171). Auckland: Auckland University of Technology.
- DigitalOcean. (2022, november 30). *DigitalOcean*. Retrieved november 30, 2022, from www.digitalocean.com: www.digitalocean.com
- Dunkerly, D. (2021). *Mike Meyers' CompTIA Security+ Certification Passport, Sixth Edition (Exam SY0-601)*. New York: McGraw-Hill Education.
- Elastic. (2022, december 12). *What Is The ELK Stack?* Retrieved december 12, 2022, from www.elastic.co: <https://www.elastic.co/what-is/elk-stack>
- Google LLC. (2022, november 10). *What is IaaS?* Retrieved november 10, 2022, from cloud.google.com: <https://cloud.google.com/learn/what-is-iaas>
- Jhummarwal, D. A., Panchal, M., Potdar, D. M., & Vadaviya, D. (2019). Types of Honeypot and Comparative Study Based on Level of Interaction Using Single Honeypot. *JETIR* (pp. 351-357). Ahmedabad, Gujarat, India: GTU School of Engineering & Technology.
- Lassen, S. M., & Olsen, N. V. (2019). *Projektledeelse*. København K: Trojka | Gads Forlag.
- Mahmoud, R.-V., & Pedersen, J. M. (2019). *Deploying a University Honeypot*. Aalborg: Aalborg University.
- MITRE. (2022, februar 28). *A Practical Guide To Adversary Engagement*. Retrieved november 24, 2022, from engage.mitre.org: <https://engage.mitre.org/wp-content/uploads/2022/04/EngageHandbook-v1.0.pdf>

MITRE. (2022, november 16). *Cybersecurity*. Retrieved november 16, 2022, from www.mitre.org:

<https://www.mitre.org/focus-areas/cybersecurity>

MITRE. (2022, februar 28). *Starter Kit Contents*. Retrieved november 10, 2022, from

<https://engage.mitre.org/starter-kit/>: <https://engage.mitre.org/starter-kit/>

Obbayi, L. (2019, december 11). *MITRE ATT&CK: Exploiting a public-facing application*. Retrieved november 22, 2022, from <https://resources.infosecinstitute.com/>:

<https://resources.infosecinstitute.com/topic/mitre-attck-exploiting-a-public-facing-application/>

Oosterhof, M. (2022, november 16). *Installing Cowrie in seven steps*. Retrieved november 16, 2022, from

[cowrie.readthedocs.io](https://cowrie.readthedocs.io/en/latest/INSTALL.html): <https://cowrie.readthedocs.io/en/latest/INSTALL.html>

Radigan, D. (n.d.). *What is kanban?* Retrieved november 12, 2022, from www.atlassian.com:

<https://www.atlassian.com/agile/kanban>

Splunk. (2022, november 16). *SPL*. Retrieved november 16, 2022, from docs.splunk.com:

<https://docs.splunk.com/Splexicon:SPL>

Splunk. (2022, december 12). *Splunk products*. Retrieved december 12, 2022, from www.splunk.com:

https://www.splunk.com/en_us/products.html

TryHackMe. (2022, november 10). *MITRE*. Retrieved november 10, 2022, from www.tryhackme.com:

<https://tryhackme.com/room/mitre>

Valeros, V. (2022, oktober 8). *Stratosphere Laboratory*. Retrieved from <https://www.stratosphereips.org/>:

<https://www.stratosphereips.org/blog/2020/10/10/installing-t-pot-honeypot-framework-in-the-cloud>

13 Figurliste

Figur 1: Cowrie.....	1
Figur 2: T-POT CPU.....	8
Figur 3: MITRE Engage: matrix	12
Figur 4: Topologi v1.....	14
Figur 5: Topologi v2 (endelig)	15
Figur 6: Fordele/ulemper v. honeypots	18
Figur 7: GCP - firewall.....	20
Figur 8: sudo systemctl status ssh	22
Figur 9: Virtuelt miljø aktiveres	23
Figur 10: iptables -t nat -nvL.....	24
Figur 11: netstat -tanpl	25
Figur 12: Splunk external IP.....	26
Figur 13: Splunk app management.....	27
Figur 14: Custom port @ GCP.....	27
Figur 15: Phone home.....	28
Figur 16: Events i Splunk	29
Figur 17: PuTTY ssh	29
Figur 18: Forsøgt login	30
Figur 19: root:admin godkendt.....	30
Figur 20: Telnet-forbindelse.....	30
Figur 21: uname -a sammenligning	31
Figur 22: Splunk dashboard.....	32
Figur 23: Top attacker commands	33
Figur 24: Cowrie architecture.....	34
Figur 25: Cowrie kernel	34
Figur 26: Cowrie PowerShell	35
Figur 27: Top countries/cities.....	36
Figur 28: Kort over forbindelsesforsøg	37
Figur 29: Top credentials.....	38
Figur 30: Kanban-board.....	40